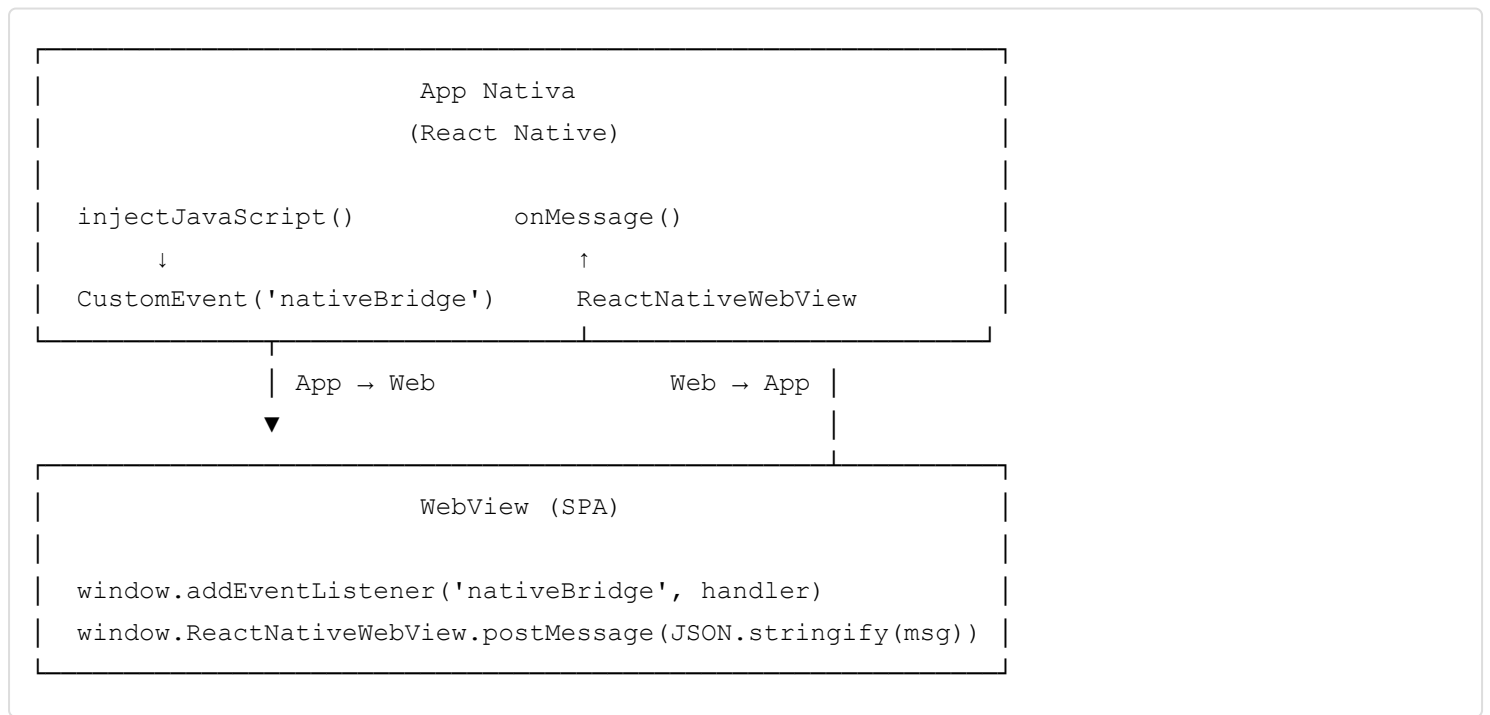


Bridge Nativo — WebView Blu Benefits

Presentación técnica para el equipo web

¿Qué es el Bridge?

El **bridge** es el canal de comunicación entre la app nativa (React Native) y la SPA embebida en el WebView. Permite que ambas partes intercambien datos de forma estructurada y segura, sin depender de `window.postMessage` ni iframes.



● **Nunca uses** `window.postMessage`, `window.parent.postMessage` ni `window.addEventListener('message', ...)`. Esos canales no forman parte del bridge oficial y serán ignorados por la app.

Estructura de todo mensaje

```
interface BridgeMessage<T = unknown> {
  type: string // nombre del evento — siempre SCREAMING_SNAKE_CASE
  payload: T // datos del evento
  [secretKey: string]: unknown // secreto del bridge (solo en mensajes Web → App)
  meta?: {
    timestamp: number // epoch ms
    version: '1.0'
    requestId?: string // UUID para trazabilidad
  }
}
```

Bridge Secret — Seguridad del canal

Antes de que tu código ejecute, la app inyecta estos valores en `window`:

```
window.__BLU_WEBVIEW__ = {
  platform: 'mobile',
  version: '1.0',
  capabilities: ['SESSION_INIT', 'LOGOUT_INACTIVITY'],
}

window.__BLU_BRIDGE_SECRET__ = 'MIIBIjAN...' // valor del secreto
window.__BLU_BRIDGE_SECRET_KEY__ = '_bluSecret' // nombre de la propiedad
```

¿Para qué sirve el secreto?

Todo mensaje que el web envíe a la app **debe incluir el secreto** como propiedad de primer nivel. Si no está presente o es incorrecto, la app descarta el mensaje silenciosamente.

```

| Mensaje Web → App (con secreto)
|
| {
|   "type": "WEBVIEW_READY",
|   "payload": {},
|   "_bluSecret": "MIIBIjAN...", ← lo agrega el helper automáticamente
|   "meta": { "timestamp": ... }
| }
|
```

Regla de oro: nunca almacenes el secreto en `localStorage`, `sessionStorage`, `cookies` ni logs. Úsalo solo en memoria.

Helper base (copia y usa en tu proyecto)

```
// src/bridge/nativeBridge.js

/** Detecta si la SPA está corriendo dentro del WebView nativo */
export const isInsideNativeApp = () =>
  typeof window !== 'undefined' &&
  !!window.__BLU_WEBVIEW__ &&
  !!window.ReactNativeWebView

/**
 * Envía un mensaje a la app nativa.
 * Agrega el secreto automáticamente — no necesitas hacerlo tú.
 */
```

```

export function sendToNative(type, payload = {}, meta = {}) {
  if (!isInsideNativeApp()) return

  const secretKey = window.__BLU_BRIDGE_SECRET_KEY__
  const secret    = window.__BLU_BRIDGE_SECRET__

  window.ReactNativeWebView.postMessage(JSON.stringify({
    type,
    payload,
    [secretKey]: secret,
    meta: { timestamp: Date.now(), version: '1.0', ...meta },
  }))
}

/**
 * Escucha mensajes provenientes de la app nativa.
 * Devuelve una función de limpieza – úsala en el return de useEffect.
 */
export function onNativeMessage(handler) {
  const listener = (event) => {
    try {
      const msg = event.detail
      if (!msg || typeof msg.type !== 'string') return
      handler(msg)
    } catch (e) { /* log con tu herramienta de observabilidad */ }
  }
  window.addEventListener('nativeBridge', listener)
  return () => window.removeEventListener('nativeBridge', listener)
}

/** Reporta un error crítico a la app nativa */
export function reportErrorToNative(code, message) {
  sendToNative('WEBVIEW_ERROR', { code, message })
}

```

Eventos

SESSION_INIT

Dirección: App → WebView

App Nativa (emisor) ——— SESSION_INIT —————> WebView (SPA) (receptor)

Quién emite App Nativa, automáticamente después de recibir `WEBVIEW_READY`

Quién recibe La SPA (tu aplicación web)

Canal `window.addEventListener('nativeBridge', ...) → event.detail`

¿Cuándo? Una vez por sesión, tras el handshake inicial

Payload que recibirás:

```
{
  "type": "SESSION_INIT",
  "payload": {
    "cedula": "1711159713",
    "coordenadas": {
      "latitud": -0.1807,
      "longitud": -78.4845
    },
    "deeplink": "campaign-info/MKT23454"
  },
  "meta": { "timestamp": 1746648000000, "version": "1.0" }
}
```

`coordenadas` contiene las coordenadas de Quito por defecto si el usuario no otorgó permisos de ubicación. `deeplink` puede venir vacío o indefinido — en ese caso, mantén la ruta por defecto.

Cómo manejarlo:

```
import { onNativeMessage } from './bridge/nativeBridge'

useEffect(() => {
  const off = onNativeMessage((msg) => {
    if (msg.type !== 'SESSION_INIT') return

    const { cedula, coordenadas, deeplink } = msg.payload

    // 1. Guardar en tu store / contexto
    setSession({ cedula, coordenadas })

    // 2. Navegar si viene deeplink
    if (deeplink) navigate(`/${deeplink}`)
  })

  return off // limpieza automática al desmontar
}, [])
```

LOGOUT_INACTIVITY

Dirección: App → WebView

App Nativa (emisor) — LOGOUT_INACTIVITY —> WebView (SPA) (receptor)

Quién emite App Nativa, cuando el timer de inactividad expira

Quién recibe La SPA (tu aplicación web)

Canal `window.addEventListener('nativeBridge', ...) → event.detail`

¿Cuándo? Cada vez que la sesión nativa caduca por inactividad

Payload que recibirás:

```
{
  "type": "LOGOUT_INACTIVITY",
  "payload": { "reason": "INACTIVITY_TIMEOUT" },
  "meta": { "timestamp": 1746648300000, "version": "1.0" }
}
```

Cómo manejarlo:

```
import { onNativeMessage } from './bridge/nativeBridge'

useEffect(() => {
  const off = onNativeMessage((msg) => {
    if (msg.type !== 'LOGOUT_INACTIVITY') return

    // 1. Limpiar estado sensible: cédula, coordenadas, formularios
    clearSession()

    // 2. Mostrar vista neutra — la app cerrará el WebView en breve
    showSessionExpiredScreen()

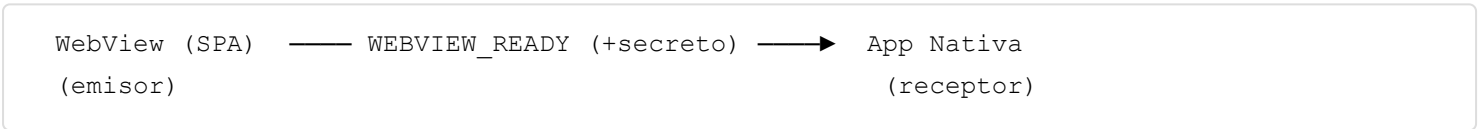
    // NO intentes navegar al login — la app nativa lo maneja
  })

  return off
}, [])
```

La app cerrará el WebView inmediatamente después de emitir este evento. Solo tienes un instante para limpiar el estado.

WEBVIEW_READY

Dirección: WebView → App



Quién emite La SPA, una vez montada y lista para recibir datos

Quién recibe App Nativa

Canal window.ReactNativeWebView.postMessage(...)

¿Cuándo? Una sola vez, al montar la SPA

Regla crítica: suscríbete a onNativeMessage **antes** de emitir WEBVIEW_READY . Si lo emites primero, perderás el SESSION_INIT .

La app espera este evento por **8 segundos**. Si no llega, muestra una pantalla de error nativa.

Cómo emitirlo:

```
import { sendToNative, onNativeMessage } from './bridge/nativeBridge'

useEffect(() => {
  // Paso 1: suscribirse primero
  const off = onNativeMessage(handleNativeMessage)

  // Paso 2: avisar que el web está listo
  // El secreto se agrega automáticamente por el helper
  sendToNative('WEBVIEW_READY')

  return off
}, [])
```

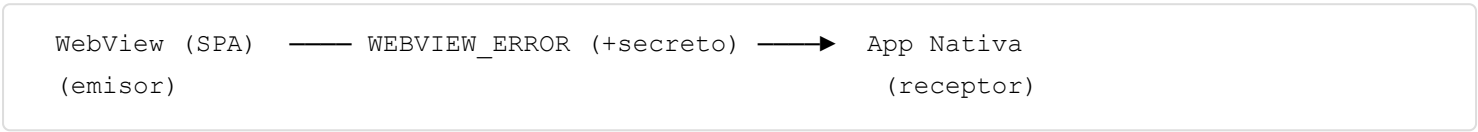
Mensaje que envía el helper a la app:

```
{
  "type": "WEBVIEW_READY",
  "payload": {},
  "_bluSecret": "MIIBIjAN...",
  "meta": { "timestamp": 1746648000000, "version": "1.0" }
}
```

Consecuencia: la app cancela el timer de 8 s y envía `SESSION_INIT` inmediatamente.

WEBVIEW_ERROR

Dirección: WebView → App



Quién emite La SPA, cuando detecta un error crítico irrecuperable

Quién recibe App Nativa

Canal `window.ReactNativeWebView.postMessage(...)`

¿Cuándo? Solo ante errores críticos que el usuario no puede resolver desde el web

Cuándo usarlo vs cuándo no:

Usar <code>WEBVIEW_ERROR</code>	No usar
Fallo al cargar recursos esenciales	Errores de validación de formulario
API crítica sin recuperación	Errores 404 de recursos secundarios
Estado corrupto que impide continuar	Timeouts recuperables con reintentos

Cómo emitirlo:

```
import { reportErrorToNative } from '../bridge/nativeBridge'

// Opción A — en un catch
try {
  await loadCriticalData()
} catch (err) {
  reportErrorToNative('CRITICAL_LOAD_FAILED', err.message)
  // A partir de aquí la app toma el control — no hagas nada más
}

// Opción B — en un error boundary de React
class BridgeErrorBoundary extends React.Component {
  componentDidCatch(error) {
    reportErrorToNative('REACT_CRASH', error.message)
  }
}
```

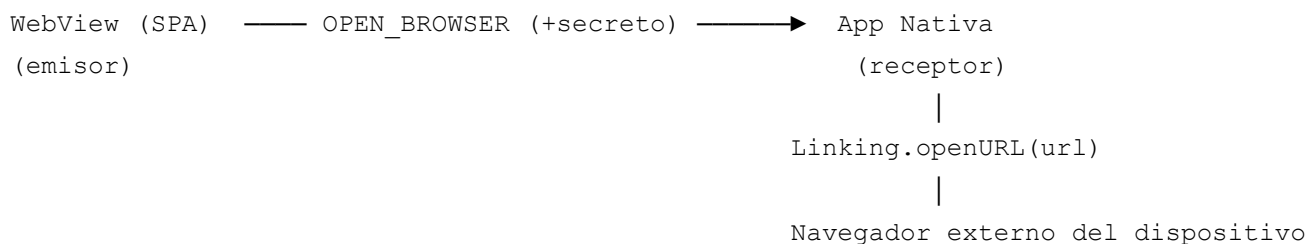
Mensaje que envía el helper a la app:

```
{
  "type": "WEBVIEW_ERROR",
  "payload": {
    "code": "CRITICAL_LOAD_FAILED",
    "message": "No se pudo cargar el módulo de beneficios"
  },
  "_bluSecret": "MIIBIjAN...",
  "meta": { "timestamp": 1746648100000, "version": "1.0" }
}
```

Consecuencia: la app muestra inmediatamente una pantalla de error nativa con botón **Reintentar**. Al reintentar, el WebView se recarga desde cero → el flujo comienza nuevamente desde `WEBVIEW_READY`.

OPEN_BROWSER

Dirección: WebView → App



Quién emite La SPA, cuando necesita abrir una URL en el navegador externo

Quién recibe App Nativa

Canal `window.ReactNativeWebView.postMessage(...)`

¿Cuándo? Ante cualquier URL que deba abrirse fuera del WebView (términos, links externos, etc.)

¿Por qué no abrirlo directamente desde el web? El WebView tiene acceso restringido a la navegación externa. La app nativa usa `Linking.openURL()` de React Native para abrir el navegador del sistema de forma confiable en iOS y Android.

Payload:

```
interface OpenBrowserPayload {
  url: string // URL completa a abrir (debe incluir protocolo: https://...)
}
```

Cómo emitirlo:

```
import { sendToNative } from './bridge/nativeBridge'

// Abre un enlace externo a través de la app nativa
function openExternalLink(url) {
  sendToNative('OPEN_BROWSER', { url })
}

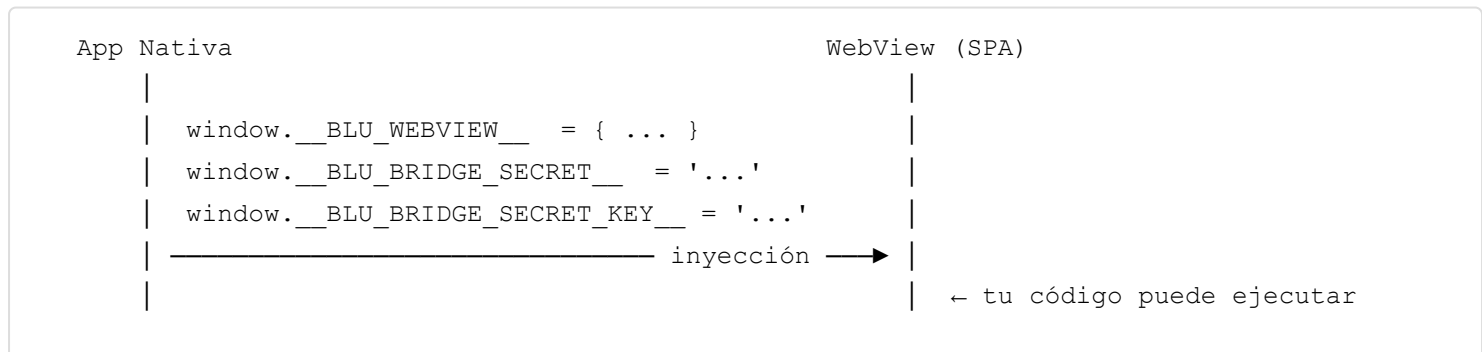
// Ejemplos de uso:
openExternalLink('https://www.banbif.com.ec/terminos-condiciones')
openExternalLink('https://maps.google.com/?q=-0.1807,-78.4845')
```

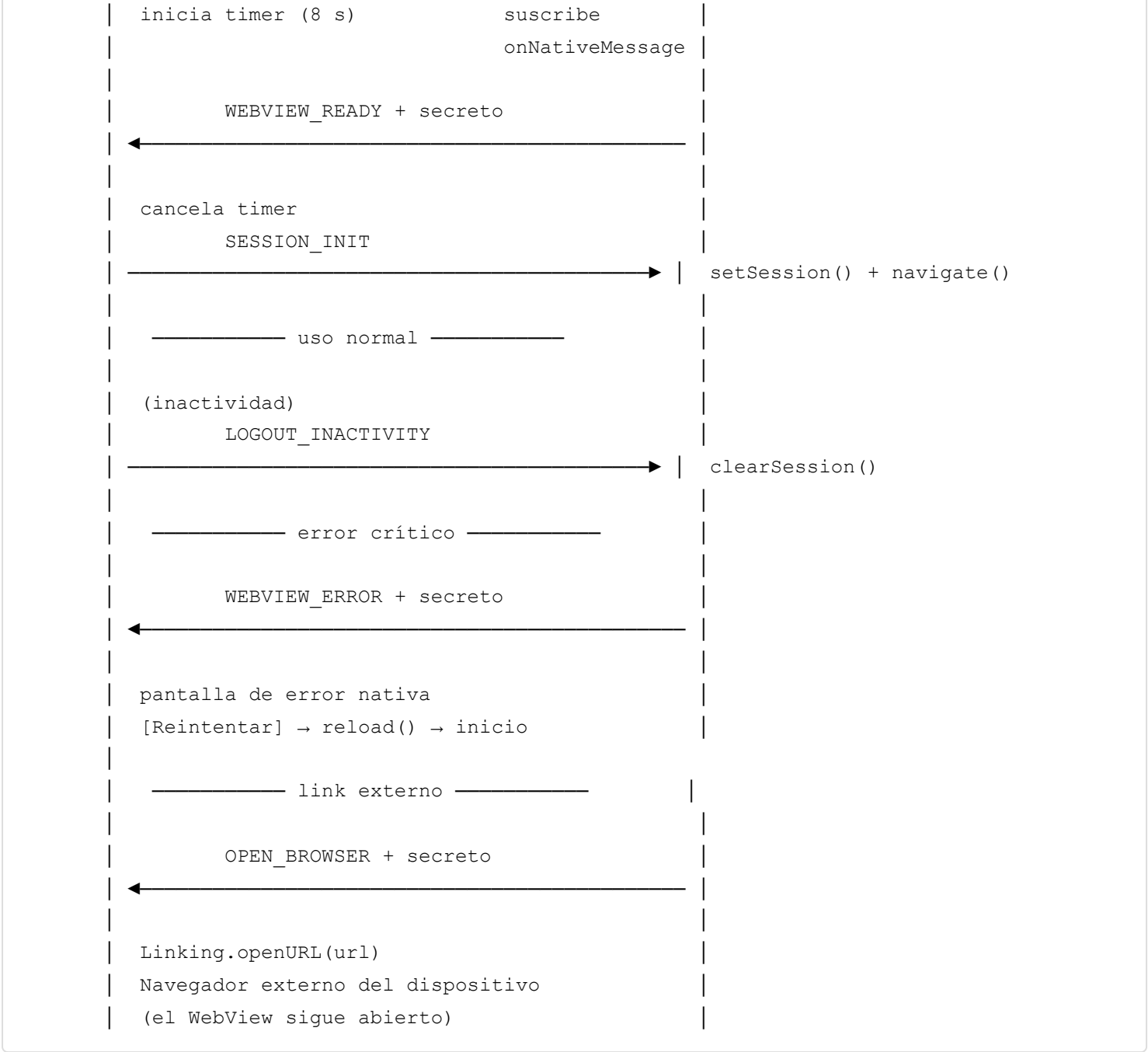
Mensaje enviado a la app:

```
{
  "type": "OPEN_BROWSER",
  "payload": {
    "url": "https://www.banbif.com.ec/terminos-condiciones"
  },
  "_bluSecret": "MIIBIjAN...",
  "meta": { "timestamp": 1746648200000, "version": "1.0" }
}
```

Consecuencia: la app llama a `Linking.openURL(url)` de React Native, que abre la URL en el navegador predeterminado del dispositivo (Safari en iOS, Chrome u otro en Android). El WebView permanece abierto en segundo plano.

Flujo completo





Resumen de eventos

Evento	Dirección	Emisor	Receptor	Secreto requerido
WEBVIEW_READY	Web → App	SPA (al montar)	App Nativa	Sí
SESSION_INIT	App → Web	App Nativa	SPA	—
LOGOUT_INACTIVITY	App → Web	App (inactividad)	SPA	—
WEBVIEW_ERROR	Web → App	SPA (error crítico)	App Nativa	Sí
OPEN_BROWSER	Web → App	SPA (link externo)	App Nativa	Sí

Checklist rápido

- [] Crear `src/bridge/nativeBridge.js` con el helper del `$Bootstrap`.
- [] Suscribirse a `onNativeMessage` **antes** de emitir `WEBVIEW_READY`.
- [] Manejar `SESSION_INIT` : guardar sesión + navegar al deeplink.
- [] Manejar `LOGOUT_INACTIVITY` : limpiar estado sensible.

- [] Usar `reportErrorToNative('WEBVIEW_ERROR', ...)` ante errores críticos irreversibles.
- [] Usar `sendToNative('OPEN_BROWSER', { url })` para abrir links externos — **nunca** `window.open()`.
- [] Nunca almacenar `window.__BLU_BRIDGE_SECRET__` en `localStorage`, `sessionStorage` ni cookies.
- [] Probar recarga del WebView: debe re-disparar `WEBVIEW_READY` → `SESSION_INIT`.